Assignment Code: DA-AG-014

Ensemble Learning | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Ensemble Learning in machine learning? Explain the key idea behind it.

Answer:

Ensemble Learning is a machine learning technique where multiple individual models, often called "base learners" or "weak learners," are combined to solve a particular computational intelligence problem.

- **Key Idea:** The core concept is that a group of weak learners can come together to form a "strong learner." By combining the predictions of multiple models through methods like weighted averaging or voting, the ensemble reduces errors such as bias and variance, leading to better predictive performance than any single constituent model could achieve on its own.

Question 2: What is the difference between Bagging and Boosting?

Answer:

Bagging (Bootstrap Aggregating) and Boosting are the two primary methodologies in ensemble learning, each taking a distinct approach to improving model performance.
Bagging works by training multiple individual models, such as Decision Trees, in a parallel and independent manner. It uses a technique called bootstrap sampling to create different subsets of the original data for each model, ensuring that every model has an equal weight in the final output. The primary goal of Bagging is to reduce the variance of the overall model, which effectively prevents overfitting and makes the predictions more stable and robust. A classic example of this approach is the Random Forest algorithm.
Example: Random Forest.

Boosting is a sequential process where models are trained one after another. The key idea behind Boosting is that each new model specifically focuses on correcting the errors or misclassifications made by the previous models in the sequence. Unlike

Bagging, where all data points have an equal chance of being selected, Boosting assigns higher weights to the observations that were difficult to predict correctly in earlier rounds. This allows the ensemble to reduce bias and convert a collection of "weak learners" into a single "strong learner". Popular implementations of this strategy include algorithms like AdaBoost and Gradient Boosting.
Example: AdaBoost, XGBoost, and Gradient Boosting.

Question 3: What is bootstrap sampling and what role does it play in Bagging methods like Random Forest?

Answer:

Bootstrap Sampling: This is a statistical resampling technique where multiple subsets of data are created by randomly selecting observations from the original dataset **with replacement**. This means some data points may appear multiple times in a single sample, while others may not appear at all.
Role in Bagging: In methods like Random Forest, each individual decision tree is trained on a different bootstrap sample. This ensures that each tree is slightly different, reducing the correlation between trees. When their results are averaged, it produces a more stable and accurate global model.

Question 4: What are Out-of-Bag (OOB) samples and how is OOB score used to evaluate ensemble models?

Answer:

OOB Samples: During bootstrap sampling, approximately one-third of the original data is left out of the training set for any specific tree. These excluded observations are called **Out-of-Bag (OOB) samples**.
OOB Score: Since these samples were not used to train a particular tree, they act as a "built-in" validation set. The OOB score is calculated by predicting the class of each sample using only the trees that did not have that sample in their bootstrap set. This provides an unbiased estimate of the model's performance without needing a separate validation dataset.

Question 5: Compare feature importance analysis in a single Decision Tree vs. a Random Forest.

Answer:

Single Decision Tree: Feature importance is calculated based on which feature provides the highest reduction in impurity (like Gini or Entropy) at a specific split. However, it is highly sensitive to small changes in data; a slight change can lead to a completely different tree structure and different "top" features.
Random Forest: Feature importance is calculated by averaging the impurity decrease across **all** trees in the forest. This makes the results much more robust and reliable, as it captures the global importance of a feature across many different data variations, reducing the bias found in a single tree.

Question 6: Write a Python program to: ● Load the

Breast Cancer dataset using

sklearn.datasets.load_breast_cancer()

- Train a Random Forest Classifier
- Print the top 5 most important features based on feature importance scores.

(*Include your Python code and output in the code box below.*)
Answer:

```
 from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

# 1. Load the Breast Cancer dataset [cite: 26]
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# 2. Train a Random Forest Classifier [cite: 27]
# Using 100 trees to ensure stable feature importance scores
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X, y)

# 3. Extract and Print the top 5 most important features
importances = pd.Series(rf_model.feature_importances_, index=data.feature_names)
top_5_features = importances.sort_values(ascending=False).head(5)

print("Top 5 Most Important Features:")
print("-" * 30)
print(top_5_features)
```

Output

```
Top 5 Most Important Features:
------------------------------
worst area              0.139357
worst concave points    0.132225
mean concave points     0.107046
worst radius            0.082848
worst perimeter         0.080850
dtype: float64
```

Question 7: Write a Python program to:

- Train a Bagging Classifier using Decision Trees on the Iris dataset
- Evaluate its accuracy and compare with a single Decision Tree

(*Include your Python code and output in the code box below.*)

Answer:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load the Iris dataset [cite: 32]
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Train a single Decision Tree [cite: 32]
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_preds = dt_model.predict(X_test)
dt_acc = accuracy_score(y_test, dt_preds)

# 4. Train a Bagging Classifier using Decision Trees as the base [cite: 32]
# Using 50 trees to create the ensemble
bagging_model = BaggingClassifier(estimator=DecisionTreeClassifier(),
                                  n_estimators=50,
                                  random_state=42)
```

```
bagging_model.fit(X_train, y_train)
bagging_preds = bagging_model.predict(X_test)
bagging_acc = accuracy_score(y_test, bagging_preds)

# 5. Evaluate and compare accuracy [cite: 32]
print(f"Single Decision Tree Accuracy: {dt_acc * 100:.2f}%")
print(f"Bagging Classifier Accuracy: {bagging_acc * 100:.2f}%")
```

Output

```
Single Decision Tree Accuracy: 100.00%
Bagging Classifier Accuracy: 100.00%
```

Question 8: Write a Python program to:

- Train a Random Forest Classifier
- Tune hyperparameters `max_depth` and `n_estimators` using GridSearchCV
- Print the best parameters and final accuracy

(*Include your Python code and output in the code box below.*)

Answer:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# 1. Load the Breast Cancer dataset [cite: 26]
data = load_breast_cancer()
X, y = data.data, data.target

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Initialize the Random Forest Classifier [cite: 36]
rf = RandomForestClassifier(random_state=42)

# 4. Define the hyperparameter grid to search
# n_estimators: Number of trees in the forest
# max_depth: The maximum depth of each tree
param_grid = {
    'n_estimators': [50, 100, 200],
```

```
    'max_depth': [None, 10, 20, 30]
}

# 5. Setup GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5, n_jobs=-1)
grid_search.fit(X_train, y_train)

# 6. Print the best parameters and final accuracy [cite: 39]
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best Cross-Validation Score: {grid_search.best_score_:.4f}")

# 7. Evaluate on the actual test set
test_accuracy = grid_search.score(X_test, y_test)
print(f"Final Test Accuracy: {test_accuracy:.4f}")
```

Output

```
Best Parameters: {'max_depth': None, 'n_estimators': 200}
Best Cross-Validation Score: 0.9626
Final Test Accuracy: 0.9649
```

Question 9: Write a Python program to:

- Train a Bagging Regressor and a Random Forest Regressor on the California Housing dataset
- Compare their Mean Squared Errors (MSE)

(*Include your Python code and output in the code box below.*)

Answer:

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# 1. Load the California Housing dataset
data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2,
random_state=42)

# 2. Train a Bagging Regressor
```

```
# Using 50 base estimators for the ensemble
bagging_reg = BaggingRegressor(n_estimators=50, random_state=42)
bagging_reg.fit(X_train, y_train)
bag_mse = mean_squared_error(y_test, bagging_reg.predict(X_test))

# 3. Train a Random Forest Regressor
# Random Forest adds extra randomness by selecting random features at each split
rf_reg = RandomForestRegressor(n_estimators=50, random_state=42)
rf_reg.fit(X_train, y_train)
rf_mse = mean_squared_error(y_test, rf_reg.predict(X_test))

# 4. Compare their Mean Squared Errors (MSE)
print(f"Bagging Regressor MSE: {bag_mse:.4f}")
print(f"Random Forest Regressor MSE: {rf_mse:.4f}")
```

Output

```
Bagging Regressor MSE: 0.2573
Random Forest Regressor MSE: 0.2573
```

Question 10: You are working as a data scientist at a financial institution to predict loan default. You have access to customer demographic and transaction history data.

You decide to use ensemble techniques to increase model performance.

Explain your step-by-step approach to:

- Choose between Bagging or Boosting
- Handle overfitting
- Select base models
- Evaluate performance using cross-validation
- Justify how ensemble learning improves decision-making in this real-world context.

(*Include your Python code and output in the code box below.*)

Answer:

Working as a data scientist to predict loan defaults requires a balance between high accuracy and interpretability. Here is the step-by-step approach:

Choose between Bagging or Boosting: I would choose **Boosting** (such as XGBoost or LightGBM). In financial datasets, loan defaults are often rare (imbalanced data). Boosting is more effective here because it sequentially focuses on the "hard-to-classify" instances (the defaulters), whereas Bagging treats all instances with equal importance.

Handle Overfitting: To ensure the model generalizes well to new customers, I would use **Regularization** (L1 or L2) and tune hyperparameters like max_depth to limit tree growth. I would also implement "early stopping," which halts training once the performance on a validation set stops improving.

Select Base Models: I would select **Decision Trees** as the base models. They are ideal for this context because they can handle both numerical data (transaction history) and categorical data (demographics) without requiring extensive feature scaling.

Evaluate Performance using Cross-Validation: I would use **Stratified K-Fold Cross-Validation**. This ensures that each "fold" of the data contains a representative proportion of both defaulters and non-defaulters, providing a more reliable estimate of the model’s real-world performance.

Justification of Ensemble Learning: Ensemble learning improves decision-making by reducing the risk associated with a single model's bias. In a financial institution, this multi-model approach leads to a more robust risk assessment, helping to minimize financial losses from unpaid loans while ensuring that creditworthy customers are not unfairly rejected.